Firjan SENAI

Técnico

JavaScript

CONCEITOS • VARIÁVEIS • ESTRUTURAS CONDICIONAIS • LAÇOS DE REPETIÇÃO • ALGORITMOS



2025



Sumário

1. **Introdução à Lógica de Programação**
   O que é lógica de programação e por que aprender
2. **Tipos de Dados e Variáveis**
   Números, textos, booleanos e declaração de variáveis
3. **Operadores e Expressões**
   Aritméticos, relacionais e lógicos
4. **Estruturas Condicionais**
   Decisões com if, else e switch
5. **Laços de Repetição**
   Repetição com for, while e do...while
6. **Algoritmos**
   Representação e construção de soluções passo a passo
7. **Funções e Modularização**
   Organização do código em blocos reutilizáveis



01

Introdução à JavaScript

Desenvolvimento de Sistemas - JavaScript

Instrutor: Vitor Hugo Almeida de Sousa

JavaScript

JavaScript é uma linguagem de programação leve, interpretada e muito popular, especialmente no desenvolvimento web. Originalmente criada para tornar as páginas da internet mais interativas, ela hoje é amplamente usada tanto no frontend (interface do usuário) quanto no backend (lógica do servidor), graças a plataformas como o Node.js. Por sua versatilidade e ampla adoção, aprender JavaScript é um passo importante para quem deseja entrar no mundo da programação.

No dia a dia, o JavaScript está presente em diversos lugares, como:

- Sites e sistemas web interativos (botões, menus, formulários dinâmicos)
- Aplicativos de celular (usando frameworks como React Native)
- Plataformas de streaming, redes sociais e e-commerces
- Painéis administrativos e sistemas corporativos
- Funcionalidades em tempo real, como chats e atualizações automáticas de conteúdo

JavaScript

Importância do JavaScript

JavaScript é uma das linguagens mais importantes da programação moderna, essencial para o desenvolvimento de aplicações interativas e dinâmicas na Web.

Atenção



Hoje em dia, o JavaScript é tão essencial para a web que, ao desativá-lo no navegador, muitos sites simplesmente deixam de funcionar corretamente. Elementos interativos como menus, botões, formulários, notificações e atualizações em tempo real dependem do JavaScript para funcionar. Sem ele, páginas podem não carregar conteúdos dinâmicos, exibir mensagens de erro ou até impedir completamente a navegação. Isso mostra como o JavaScript se tornou fundamental para a experiência do usuário na internet moderna.

JavaScript

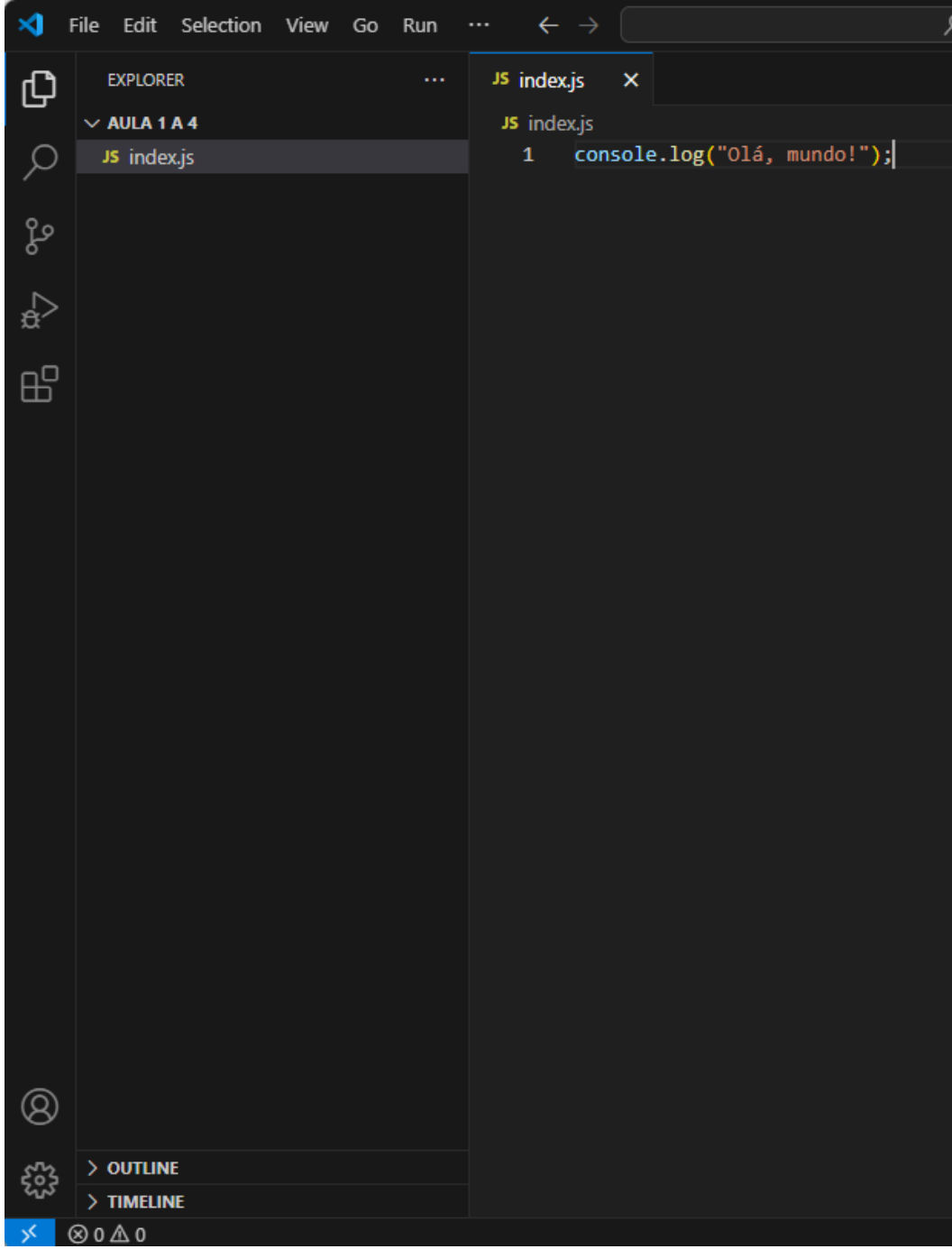
Executando primeira aplicação no VS Code

Iniciando sua jornada prática na programação com VS Code

- Escrevendo o primeiro código JavaScript
- Executando o código no terminal do VS Code
- Entendendo a saída no terminal

Exercícios

- Crie uma pasta na Área de Trabalho chamada www
- Abra a pasta www no VS Code
- Dentro do VS Code, crie uma pasta Aula 1
- Crie um arquivo chamado index.js dentro da pasta Aula 1
- Crie um código “Olá, Mundo” em JavaScript
- Abra o Terminal e execute o código no VS Code



Fontes de imagem – imagem de criação própria

JavaScript - Comentários

Assim como qualquer outra linguagem de programação, o JavaScript também permite o uso de comentário no meio do código.

Saiba Mais!



Em JavaScript, os comentários são usados para explicar partes do código, facilitar a leitura e ajudar na organização, sem interferir na execução do programa. Existem dois tipos: o comentário de uma linha, feito com //, e o comentário de múltiplas linhas, feito com /* */. Eles são úteis para anotar ideias, descrever a função de trechos específicos ou desativar temporariamente partes do código durante testes.

```
1  // O código a seguir conta uma história
2  console.log("Uma história legal")
3
4  /* Além disso, eu posso comentar
5  uma parte que possui várias linha */
6
```

JavaScript - Variáveis

Uma variável é como uma “caixinha” onde podemos guardar informações para usar no nosso programa. Essas informações podem ser textos, números, ou outros tipos de dados. Ao dar um nome para essa caixinha, conseguimos acessá-la sempre que precisarmos daquele valor. Por exemplo, podemos criar uma variável chamada nome para guardar o nome de uma pessoa, e depois mostrar esse nome na tela. As variáveis são muito importantes na programação porque nos ajudam a armazenar e manipular dados de forma simples e organizada.

Agora que você já sabe o que é uma variável, é importante conhecer algumas regrinhas para usá-las corretamente:

- O nome de uma variável não pode começar com número, não pode conter espaços ou caracteres especiais (como @, #, %, etc.)
- Não pode ser igual a palavras reservadas do JavaScript, como let, if ou function
- Usar nomes que façam sentido com o que está sendo guardado, como nome, idade ou precoProduto
- Variáveis podem guardar diferentes tipos de dados, como números (ex: 20), textos (ex: "Olá"), ou valores booleanos (verdadeiro ou falso, como true ou false)
- Toda vez que quisermos criar uma variável, precisamos começar com uma palavra especial do JavaScript. Vamos usar a palavra let para isso.

Firjan SENAI

JavaScript

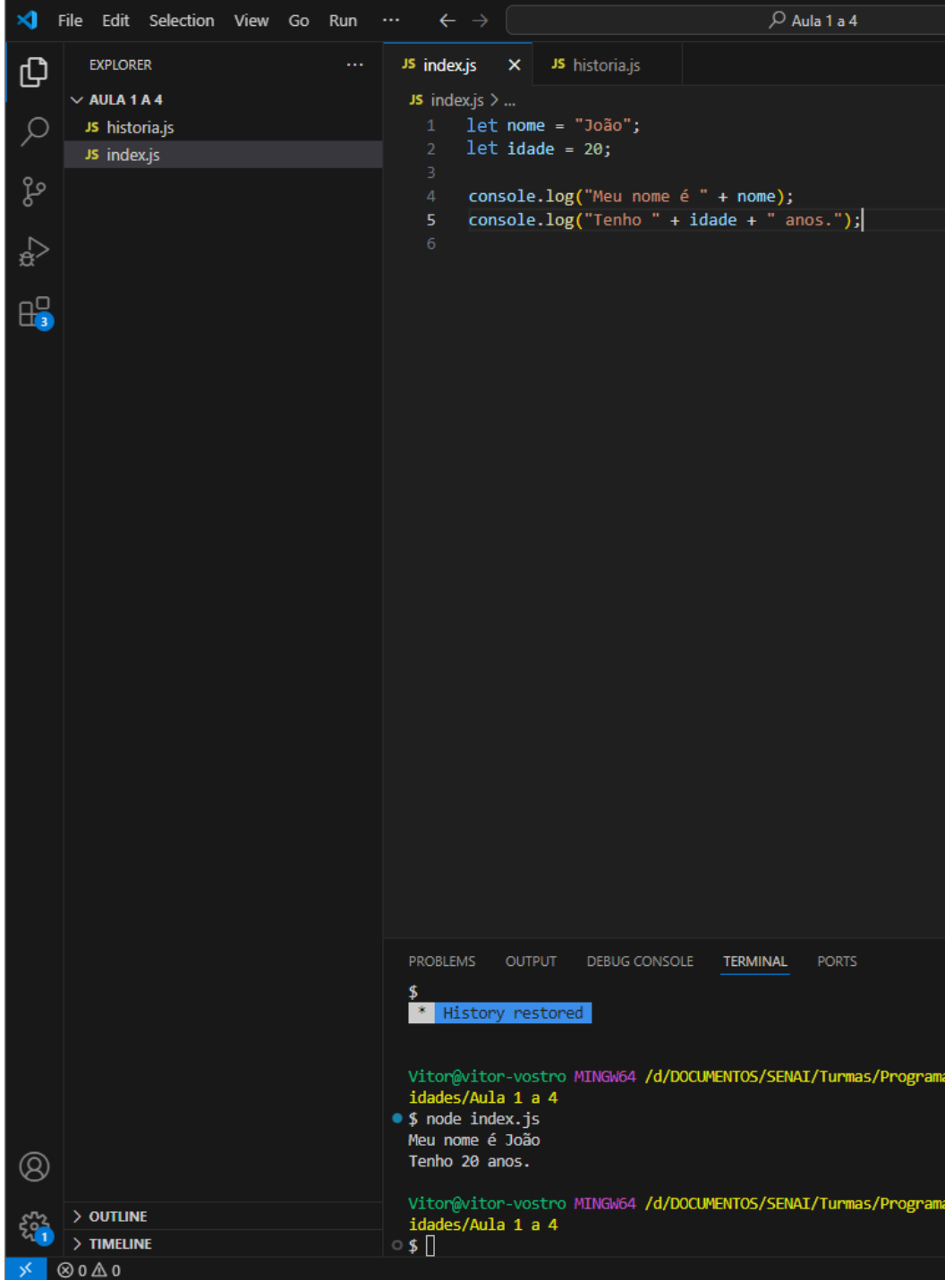
Usando Variáveis

Objetivo: entender declaração de variáveis e concatenação.

- Como declarar variáveis
- Como exibir valores de variáveis
- Como concatenar variáveis

Exercícios

- Abra a pasta www no VS Code;
- Crie um arquivo chamado variaveis.js dentro da pasta Aula 1;
- Crie um código na imagem ilustrada ao lado;
- Abra o Terminal e execute o código no VS Code e observe o resultado.



JavaScript - Tipos de Variáveis

Em JavaScript, os tipos de variáveis mais comuns são: number, para números inteiros ou decimais; string, para textos entre aspas; boolean, que representa valores lógicos como true ou false; undefined, que indica uma variável declarada mas sem valor atribuído; null, que representa ausência intencional de valor; object, usado para armazenar dados em pares chave:valor; e array, que é uma lista ordenada de valores.

Em JavaScript, uma variável pode armazenar diferentes tipos de dados. Os principais tipos são:

- Number (número)

```
let idade = 26
let preco = 9.99
```

```
console.log(2+2) //4
```

- String (texto)

```
let nome = "Maria"
let saudacao = `Olá, ${nome}!`
```

```
console.log(2+2) //4
```

- Boolean (lógico)

```
let estaLogado = true;
let temPermissao = false;
```

Firjan SENAI

JavaScript

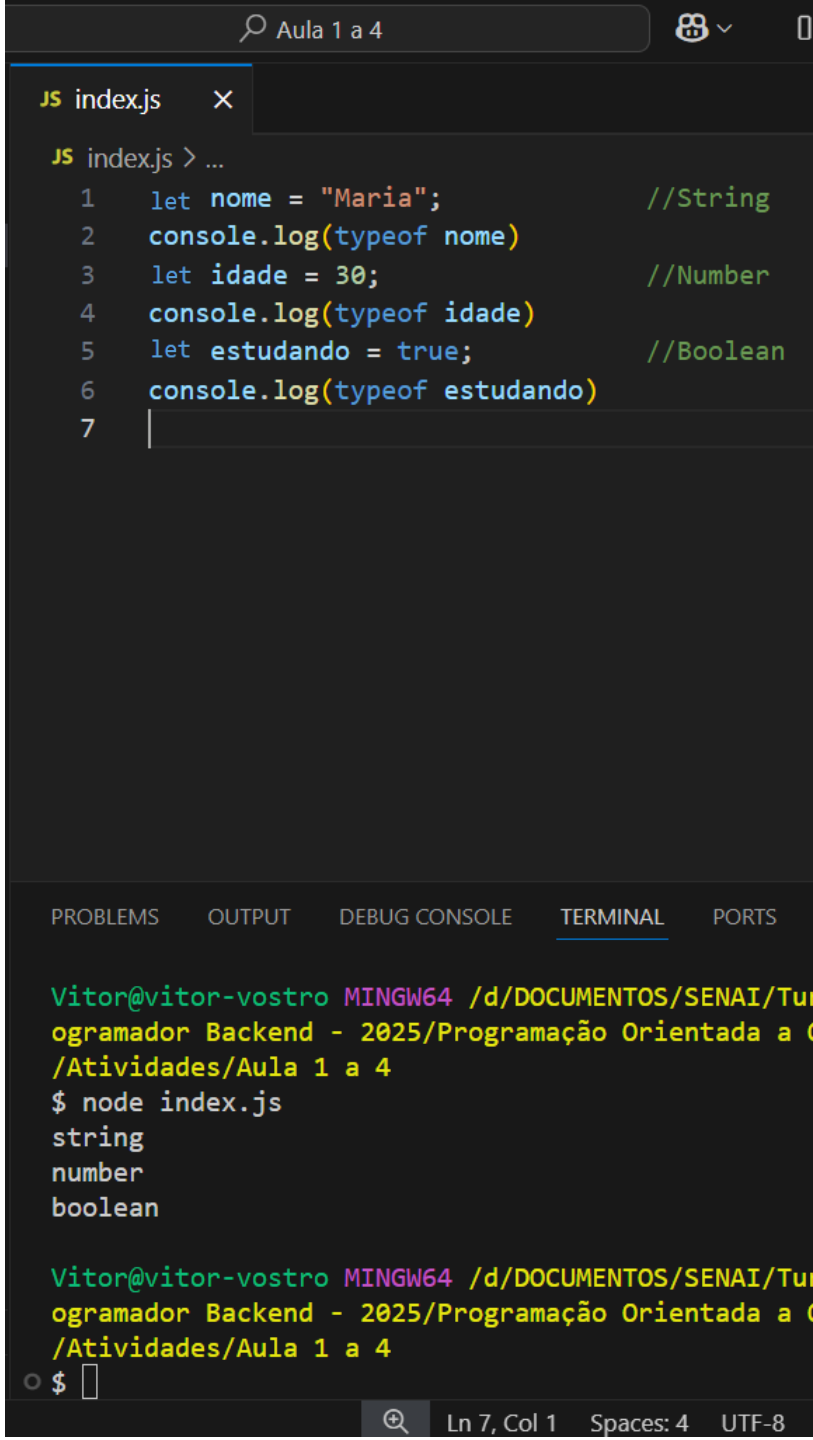
Descobrindo o tipo da variável

Objetivo: entender os tipos de variáveis.

- Como descobrir tipo de variáveis
- Como exibir esse resultado no console

Exercícios

- Abra a pasta www no VS Code;
- Crie o arquivo tipos_variáveis.js e faça o código ao lado;



JavaScript - Operadores

Operadores em JavaScript são símbolos ou palavras que permitem realizar operações sobre valores e variáveis, como atribuição, comparação, aritmética, lógica e muito mais. Eles são fundamentais para construir expressões que manipulam dados, controlam fluxos de decisão e definem comportamentos no código. Entre os principais operadores estão os aritméticos (+, -, *, /), de atribuição (=), os de comparação (==, !=, !=, >, <), os lógicos (&&, ||, !) e os relacionais, que ajudam a avaliar condições em estruturas como if, while e for.

O que vamos trabalhar no momento são os operadores de comparação e lógicos. Você vai perceber que estes sempre retornarão um valor booleano (true, false). Veja a seguir como utilizá-los em JavaScript:

- Igualdade de valor (==)
- Diferença de valor (!=)
- Maior que (>)
- Menor que (<)
- Maior ou igual a
- Menor ou igual a

```
console.log(5 == 5);
console.log(5 != 5);
console.log(10 > 7);
console.log(3 < 5);
console.log(7 >= 7);
console.log(4 <= 9);
```

```
console.log(5 == 5 && 10 > 7)
console.log(5 == 4 && 10 > 7)

console.log(5 == 5 || 5 > 7)
console.log(5 == 4 || 5 > 7)
```

Igualdade: == e ===

Em JavaScript, os operadores == e === comparam dois valores. A diferença está no tratamento dos tipos: o == pode converter os valores antes de comparar, enquanto o === exige que eles tenham o mesmo tipo e o mesmo valor.

```
5 == "5"    // true: o texto "5" é convertido para número.
5 === "5"   // false: um é número e o outro é texto.
5 === 5     // true: mesmo tipo e mesmo valor.
```

Saiba Mais!



Em um sistema de produtos, os códigos "001" e "1" podem identificar itens diferentes. Porém, ao compará-los com o número 1 utilizando ==, ambos são considerados iguais:

```
console.log("001" == 1); // true
console.log("1" == 1);   // true
```

Isso acontece porque a conversão automática elimina os zeros à esquerda. Uma busca pode acabar selecionando o produto errado!

Com ===, os tipos diferentes impedem essa igualdade:

```
console.log("001" === 1);     // false
console.log("001" === "1");   // false
console.log("001" === "001"); // true
```

Mantenha um padrão para os tipos dos dados e prefira === nas comparações.

JavaScript - If else

As instruções condicionais permitem que o programa execute ou ignore certos trechos de código com base no resultado de uma expressão. Elas funcionam como pontos de decisão dentro do código, sendo chamadas também de “ramificações”. Imagine que o interpretador JavaScript percorre seu código como se fosse um caminho — quando encontra uma condicional, ele precisa escolher qual direção seguir entre várias possíveis. As próximas seções vão explicar as principais formas de usar essas instruções no JavaScript, como o if/else, e também o switch, que é usado para lidar com várias possibilidades de escolha.

Segue um exemplo de código JavaScript utilizando a estrutura if

- Sintaxe

```
if (idade >= 18) {
    console.log("Você é maior de idade.");
} else {
    console.log("Você é menor de idade.");
}
```

```
1  let numero = -3;
2
3  if (numero > 0) {
4      console.log("Número positivo.");
5  } else if (numero < 0) {
6      console.log("Número negativo.");
7  } else {
8      console.log("Zero.");
9  }
```

Firjan SENAI

JavaScript — Incremento

Como aumentar valores

O incremento permite somar valores a uma variável de forma prática e previsível, muito usado para contadores e cálculos simples.

- Incremento tradicional: Soma diretamente ao valor da variável
  - Exemplo: x = x + 1 significa “pegue o valor de x e some 1”
- Incremento específico do JavaScript: Usa o operador ++
  - Exemplo: x++ significa “aumente x de 1 em 1”
  - Também pode ser usado antes da variável: ++x
  - Faz a mesma operação que x = x + 1, porém de forma mais enxuta
- Diferença pré e pós:
  - x++: primeiro usa o valor de x, depois incrementa
  - ++x: primeiro incrementa, depois usa o valor

Use o incremento ++ para deixar seu código mais simples e legível.

```
1   // Incremento tradicional
2   let x = 5;
3   x = x + 1;
4   console.log(x); // saída: 6
5
6
7   // Incremento específico (pré-incremento)
8   let z = 5;
9   ++z;
10  console.log(z); // saída: 6
11
```

Firjan SENAI

JavaScript — Decremento

Como diminuir valores

O decremento permite subtrair valores de uma variável de maneira rápida e prática, sendo muito usado para contadores decrescentes.

- Decremento tradicional: Subtrai diretamente do valor da variável
  - Exemplo: y = y - 1 significa “pegue o valor de y e subtraia 1”
- Decremento específico do JavaScript: Usa o operador --
  - Exemplo: y-- significa “diminua y de 1 em 1”
  - Também pode ser usado antes da variável: --y
  - Faz a mesma operação que y = y - 1, mas de forma mais compacta
- Diferença pré e pós:
  - y--: primeiro usa o valor de y, depois decrementa
  - --y: primeiro decrementa, depois usa o valor

O decremento -- torna o código mais limpo e fácil de entender.

```
1  // Decremento tradicional
2  let a = 5;
3  a = a - 1;
4  console.log(a); // saída: 4
5
6  // Decremento específico (pré-decremento)
7  let c = 5;
8  --c;
9  console.log(c); // saída: 4
10
```

Firjan SENAI

Incremento e Decremento para valores maiores que 1

Como alterar variáveis de forma prática

Em JavaScript, podemos aumentar ou diminuir valores de variáveis não apenas de 1 em 1, mas de qualquer quantidade desejada, usando operadores compostos:

- Incremento tradicional:
  - x = x + 5
  - aumenta o valor de x em 5 unidades
- Incremento simplificado:
  - x += 5

- Decremento tradicional:
  - y = y - 3
  - diminui o valor de y em 3 unidades
- Decremento simplificado:
  - y -= 3

x += n → incrementa n unidades
x -= n → decrementa n unidades
mantém o código mais legível e limpo

```
1   // incremento tradicional
2   let x = 10;
3   x = x + 5; // x passa a valer 15
4   console.log(x); // saída: 15
5
6   // incremento simplificado
7   let y = 10;
8   y += 5; // y também passa a valer 15
9   console.log(y); // saída: 15
10
11  // decremento tradicional
12  let a = 20;
13  a = a - 4; // a passa a valer 16
14  console.log(a); // saída: 16
15
16  // decremento simplificado
17  let b = 20;
18  b -= 4; // b também passa a valer 16
19  console.log(b); // saída: 16
20
```

Firjan SENAI

JavaScript — Estruturas de Repetição

Como repetir comandos de forma controlada

O laço for permite executar um bloco de código várias vezes de forma prática e previsível. Sua estrutura é composta por três partes: inicialização, condição e incremento. Ele é ideal quando sabemos quantas vezes o código deve se repetir.

- Inicialização: Diz por onde o laço começa
  - Exemplo: let i = 0 significa que começamos no número 0
- Condição: Enquanto for verdadeira, o laço continua. Quando ficar falsa, o laço para.
  - Exemplo: i < 10 quer dizer: “continue enquanto i for menor que 10”.
- Atualização: Faz o contador andar para a próxima volta.
  - Exemplo: i++ significa “aumente i de 1 em 1”Evita

Use for para percorrer listas e contar repetições de forma simples

```
for (inicialização; condição; atualização) {
  // bloco de código
}

// Contar de 1 a 5
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// saída: 1, 2, 3, 4, 5

// Contar de 10 a 0, de 2 em 2
for (let i = 10; i >= 0; i -= 2) {
  console.log(i);
}
// saída: 10, 8, 6, 4, 2, 0
```

Firjan SENAI